BlackJack Program

Design Specification Document

Group 2
Team Members

Daniel Alvarado, Monisha Turan, Nikki Ong,

Sadhvik Chelmatikary, Tochi Ezeife

Revision History

Date	Revision	Description	Author
03/29/2026	1.0	Initial Version	All Members
04/01/2026	1.1	Added User Interface Design	Tochi Ezeife
04/02/2026	1.2	Edited Layout + Updated Table of Contents	Nikki Ong
04/06/2026	2.0	Edited + Finalized Class Diagram	Sadhvik Chelmatikary
04/06/2026	2.1	Edited Layout of Class Diagram	Nikki Ong
04/07/2026	3.0	Added Sequence Diagrams	Monisha Turan
04/08/2026	3.1	Edited +Added more Seq Diagrams	Monisha Turan
04/08/2026	3.2	Edited + Updated Goals & Objectives	Monisha Turan
04/08/2026	3.3	Added Sequence Diagrams	Tochi Ezeife
04/08/2026	3.4	Added a Sequence Diagram + Adjusted Layout of Class Diagram	Nikki Ong

Table of Contents

Revision History **2**
Table of Contents **3**
1. Goals & Objectives **4**
1.1 Scope 4
1.2. Overview 4
1.3. Constraints 4
2.User Interface Design: 6
3. Use Case Specifications: **7**
4. UML Use Case Diagram: **8**
5. Class Diagram: **19**
6. Sequence Diagrams: **20**

1. Goals & Objectives

This document outlines the design details for a Multiplayer Black Jack Gaming System. The purpose of this document is to describe how the system will be organized, how its components will interact, and how the design satisfies the requirements defined in the Software Requirements Specification (SRS).

1. Scope

This document defines the overall system design, including the architecture, major components, and interactions between different parts of the system. It provides a detailed representation of how the system will be implemented using object-oriented design principles.

The scope of this document includes:

- The system's client-server architecture
- The design of major modules:
    - User module
    - Server module
    - Game Logic module
    - Networking module
    - Client/GUI module
- UML diagrams including;
    - Use Case diagrams
    - Class diagrams
    - Sequence diagrams
- The interaction between players, dealers, and system components during gameplay.

2. Overview

The Multiplayer Black Jack Gaming System, is designed to simulate a real-time casino experience, allowing multiple players to simultaneously compete against a real-time dealer. Where players aim to reach a hand value of 21 as close as possible without exceeding it, while beating the dealer's hand.

3. Constraints

2.4.1 The system must be implemented entirely in Java.

2.4.2 Since the system has to operate as a Java application it means there mustn't have web-based code be used (HTML, CSS, JavaScript).

2.4.3 The system must use Java socket connections due to it being a client-server application.

2.4.4 The server has to handle client disconnections without stopping active games/other connections in the server system.

2.4.5 The server must support multiple client connections since multiple players can play at the same time.

2.4.6 The system has to correctly handle Blackjack standard rules such as an Ace valuation (it being value 1 or 11) & recognize a natural BlackJack on the first deal.

2.4.7 Players must not lose money unfairly, meaning their account balance can't drop below zero where bets must be validated by the server before each round.

2.4.8 Only registered users with valid credentials can connect to the server, since players have to be authenticated before joining a game.

2.4.9 The system must not allow players to access other player's hands (each player's cards in game) or manipulate game outcomes as the system has to be fair.

2.4.10 The system must maintain valid decks for each table.

User Interface Design:

Log In Window

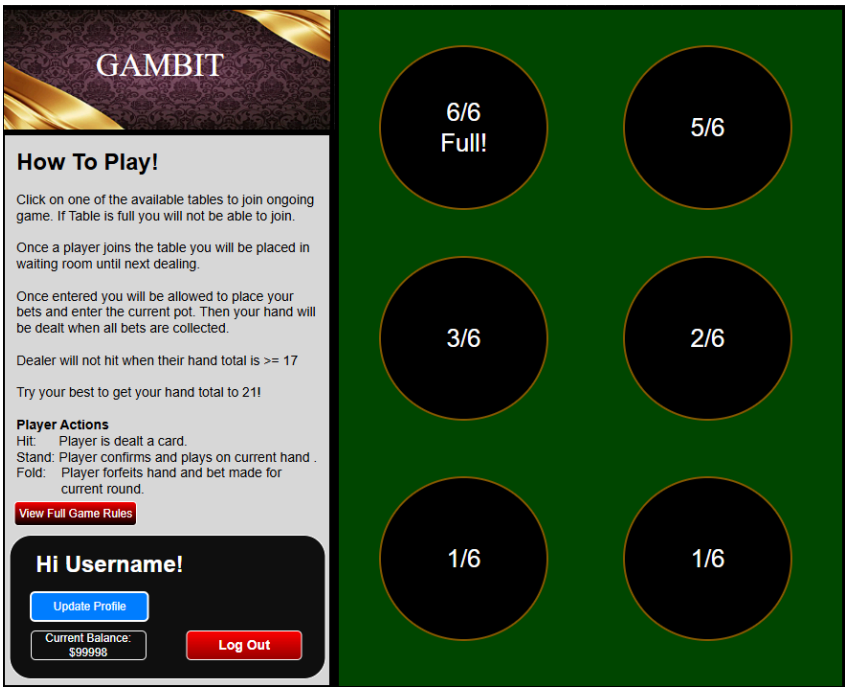
1. *Labels*: Username, Password
2. *Textfields*: Username, Password
3. *Buttons*: Sign Up, Login

Sign Up Window

1. *Labels*: Username, New Password, Confirm Password, Info for password requirements
2. *Textfields*: Username, New Password, Confirm Password
3. *Buttons*: Sign Up

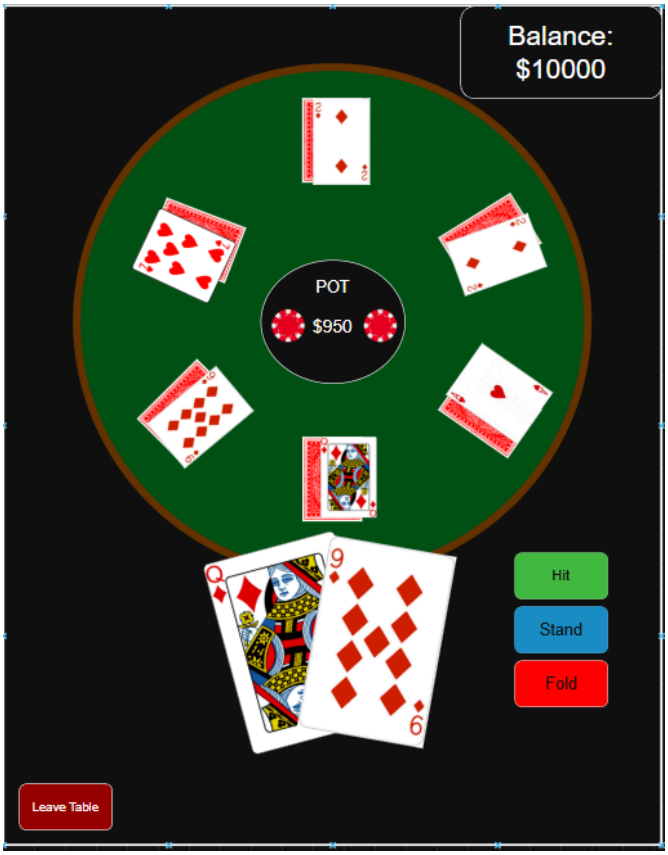
Game Tables Window

1. *Labels*: How To Play Info, Hello User, Capacity of each table (#/6, if max reached says Full!), User's balance
2. *Buttons*: Update Profile, Log Out, View Full Game Rules



In-Game Window

1. *Labels*: Dollar Amount in Pot, User Balance
2. *Buttons*: Hit, Stand, Fold, Leave Table



Use Case Specifications:

UC ID: 001

Use Case Name: Create Account

Relevant Requirements: 3.1.2.1, 4.1.1 - 4.1.3

Primary Actor: Player, Dealer

Pre-Conditions: User is not currently logged In, Client is connected to the server.

Post-Conditions: A new user account is created and stored with username, password, and starting funds.

Basic Flow/ Main Scenario:

1. User selects “Create Account” from the main menu.
2. System prompts user to enter a username and password.
3. User enter the required credentials in required format.
4. System checks username is not already in use, validates uniqueness.
5. System creates the account and confirms success.

Extensions/Alternate Flows:

1. Username already exists, system displays “Username already exists” and requests new username.
2. Password is Invalid - displays password rules and ask user to try again.

Exceptions: System fails to save account details, displays error message and returns to main page.

Related Use Cases: UC-002

Use Case ID: 002

Use Case Name: Log In

Relevant Requirements: 3.1.1.1, 3.1.2.2, 4.1.1

Primary Actor: Player/Dealer

Pre-conditions:

- Account exists.
- Client connects to the server.

Post-conditions: User is authenticated and enters the main menu/lobby.

Basic Flow or Main Scenario:

1. The user enters username and password.

2. System sends details to server and server verifies the login information.
3. System grants the access to the main menu/lobby.

Extensions or Alternate Flows:

- Invalid credentials, denies access and prompts ot retry.

Exceptions: Server is not reachable, connection error.

Related Use Cases: UC-003, UC-005.

Use Case ID: 003

Use Case Name: Deposit Funds

Relevant Requirements: 3.1.2.3, 3.1.2.5

Primary Actor: Player

Pre-conditions:

- Player must be logged in with their registered credentials

Post-conditions:

- Player's account balance is updated with increase in deposit amount.

Basic Flow or Main Scenario:

1. The player chooses the "Deposit Funds" option.
2. The system prompts the player: "Enter Amount:" to deposit.
3. The player enters the amount they want to deposit.
4. Client sends the deposit request to the server.
5. The server updates the player's account balance.
6. The system displays the updated balance to the player.

Extensions or Alternate Flows:

1. The player chooses to cancel the action of depositing.

Exceptions:

- If an invalid amount was entered (such as inputting letters), the system displays error in amount.
- Server could fail to update account balance, the system displays an error in the server.

Related Use Cases: UC-004, UC-007

Use Case ID: 004

Use Case Name: Cash Out Funds

Relevant Requirements: 3.1.2.4

Primary Actor: Player

Pre-conditions:

- The player must be logged in with their registered credentials.
- The player is currently not playing a game.

Post-conditions:

- The player's account balance is withdrawn.
- Player account balance is updated.

Basic Flow or Main Scenario:

1. The player chooses the “Cash Out” option.
2. The system displays the player’s current balance.
3. The player confirms to proceed with withdrawal.
4. The server processes the withdrawal request.
5. The system displays “Withdrawal processed” and updates the account’s balance.

Extensions or Alternate Flows:

1. The player chooses to cancel the withdrawal request.

Exceptions:

- The server could fail processing the withdrawal request, the system displays an error in the server.

Related Use Cases: UC- 003

Use Case ID: 005

Use Case Name: Player Joins Table

Relevant Requirements: 3.1.2.7, 3.1.1.3

Primary Actor: Player - Non-dealer user that will participate in game

Pre-conditions:

- Player is logged in.
- The dealer is present at the game table.

Post-conditions: Player is able to join table and dealer begins round.

Basic Flow or Main Scenario:

1. Player clicks “Play”
2. System checks for available game tables
3. Game table was found with an unoccupied seat.
4. Player joins table
5. The dealer begins the round.

Extensions or Alternate Flows:

1. A table was found, but was full (6 players occupied the 6 spots), so the system generates a new table.
2. No game tables were found, so the system generates a new table and assigns a dealer to the table.

Exceptions:

- The server could fail to find a table, the system displays an error in the server.

Related Use Cases: UC-006, UC-007

Use Case ID: 006

Use Case Name: Create New Table

Relevant Requirements: 3.1.1.9

Primary Actor: Server

Pre-conditions:

- Player attempts to join table and All tables are full.

Post-conditions:

- New table is created.
- Requesting player joins the new table.

Basic Flow or Main Scenario:

1. Server detects no available seats.
2. Creates a new table instantly.
3. Adds new table to active tables list.

Extensions or Alternate Flows:

1. If no dealer is currently available, the table created will be in “Waiting for Dealer” status. The system displays “Waiting for Dealer”.
2. If multiple players join simultaneously, the server processes a first-come-first-serve order to allocate the player to respective table slots.

Exceptions:

- Server fails to access memory to create a new table, shows error “Unable to create table Try Again”.
- Network failure during creation, player returns to the main lobby, notified of connection server.

Related Use Cases: UC-005, UC-008.

Use Case ID: 007
Use Case Name: Place Bet
Relevant Requirements: 3.1.1.4, 3.1.2.6
Primary Actor: Player
Pre-conditions:
- Player is seated on table.
- Player has sufficient funds.
- Player is logged in

Post-conditions:
- Bet is recorded for round.

Basic Flow or Main Scenario:
1. Players enter a bet amount.
2. Server verifies and validates the amount.
3. Bet is confirmed

Extensions or Alternate Flows:
1. Bet is exceeding the current balance and the server rejects it.

Exceptions: Invalid input format like any alphabets or special characters.
Related Use Cases: UC-008

Use Case ID: 008
Use Case Name: Start Round (Dealer Begins Game)
Relevant Requirements: 3.1.1.2
Primary Actor: Dealer
Pre-conditions:
- The dealer is at the table.
- At least one player is seated.
- All seated players have placed bets.

Post-conditions: Round begins and initial cards are dealt.
Basic Flow or Main Scenario:
1. Dealer selects “Start Round”.
2. The system checks that at least 1 dealer and 1 player are present.
3. verifies that all players have placed a bet.
4. Server starts the round and triggers initial dealing.

Extensions or Alternate Flows:
5. Missing dealer or player system displays “Waiting for players/dealer.”

6. One or more players who don't have a bet system displays “Waiting for bets.”

Exceptions: Server fails to initialize round, system displays error and returns table to waiting state.

Related Use Cases: UC-009

Use Case ID: 009

Use Case Name: Deal Initial Cards

Relevant Requirements: 3.1.4.2

Primary Actor: Server

Pre-conditions: Round has started. Deck is available. Bets are locked in.

Post-conditions: Each player has initial cards and dealer has initial cards, each receives two cards.

Basic Flow or Main Scenario:

1. Server shuffles the deck.
2. Server deals two cards to each player at the table.
3. Server deals two cards to the dealer (at least one visible card).
4. System updates all clients with starting hands (players only see their own hand + dealer visible card).
5. System sets turn order to the first player clockwise from the dealer.

Extensions or Alternate Flows: If deck is low on cards system resets deck and shuffles before dealing.

Exceptions: Network message fails to deliver updates, displays error.

Related Use Cases: UC-010

Use Case ID: 010

Use Case Name: Player Turn (HIT/STAND/FOLD)

Relevant Requirements: 3.1.2.8, 3.1.1.10, 3.1.3.4

Primary Actor: Player

Pre-conditions:

- Round is currently active and players are present on table.

- Player's turn in clockwise order.
- Players have placed valid bets.

Post-conditions:

- Selected actions are processed. (Hit/Stand/Fold).
- Game state is updated and Turn order progresses.

Basic Flow or Main Scenario:

1. Server notifies player of their turns and Displays action buttons: Hit/ Stand/Fold.
2. Player selects an action within a certain given period of time.
3. Client sends the selected action to server and server validates and processes the selected action.
4. The server updates game state to all the players.
5. If the player selects Stand/Fold or Busts, turn advances to the next player.

Extensions or Alternate Flows:

- Player chooses Hit: server deals one card and updates hand total.
  Player chooses Stand: player ends turn.
- Player chooses Fold: player loses bet and ends turn.
- No action in allotted turn-time: system auto-folds the player.

Exceptions:

- Server processing error.
- Network message fails to deliver updates, displays error.

Related Use Cases: UC- 009, UC-011, UC-012, UC-014

Use Case ID: **011**

Use Case Name: **Dealer Turn**

Relevant Requirements: 3.1.4.4, 3.1.15

Primary Actor: Dealer

Pre-conditions:

- Round is active, all players have finished their turns. (hit/stand/fold)
- Dealer already has two cards from the initial deal.

Post-conditions:

- The dealer finishes their turn, and the final hand is shown to all players.
- The system is ready to determine winners and payout.

Basic Flow or Main Scenario:

1. The system reveals the dealer's hidden cards.
2. Calculates dealer's hand total.
3. If the dealer total is less than 17, the dealer draws one card.
4. The system updates the dealer's hand total after each card.
5. The dealer stops drawing when the hand total reaches 17 or higher.
6. System updates the dealer's final hand to all players

Extensions or Alternate Flows:

- If the dealer's hand total is greater than 21, the dealer busts and stops the round.
- If an Ace is in the dealer's hand, the system counts it as 1 or 11.

Exceptions:

- Server fails to draw/update cards , system shows error and stops the round.
- Network message fails to deliver updates, displays error.

Related Use Cases: UC-009, UC-010, UC–12

Use Case ID: 012

Use Case Name: Determine Winners / Payout
Relevant Requirements: 3.1.1.6, 3.1.3.6
Primary Actor: Server
Pre-conditions: All players have completed their turns OR have busted/folded. The dealer has completed the dealer turn.
Post-conditions: Winners are determined and player balances are updated.

Basic Flow or Main Scenario:

1. Server confirms all players are done acting.
2. Dealer completes dealer turn (draw until 17+).
3. The server compares each player's final hand to the dealer's hand.
4. The server determines the outcome (win/lose/push/blackjack) for each player.
5. The server updates each player's balance based on the result.

6. The system displays results to all players and prompts replay/next round.

Extensions or Alternate Flows: Player folded- player automatically loses bet for that round.

Exceptions: Server fails during payout update, system displays error and prevents starting new round until resolved.

Related Use Cases: UC-013

Use Case ID: 013

Use Case Name: Replay Round/ Return to Menu

Relevant Requirements: 3.1.1.11

Primary Actor: Player

Pre-conditions:

- The round is completed.

Post-conditions:

- The player stays for the next round OR exits to the menu.

Basic Flow or Main Scenario:

1. System prompts replay option.
2. The player selects replay or exit.

Extensions or Alternate Flows:

- The player does not select an option.
- Replays with insufficient funds.

Exceptions: Network interruption, server fails to reset the round status.

Related Use Cases: UC- 007, UC-008, UC-005, UC-014

Use Case ID: 014

Use Case Name: Handle Player/Dealer Disconnect

Relevant Requirements: 3.1.3.7

Primary Actor: Server

Pre-conditions:

- The player loses connection.

Post-conditions:

- The server continues to run.
- Player removed or auto-folded.

Basic Flow or Main Scenario:

1. Server detects disconnection.
2. Server removes player from active list.
3. Game run continuously.

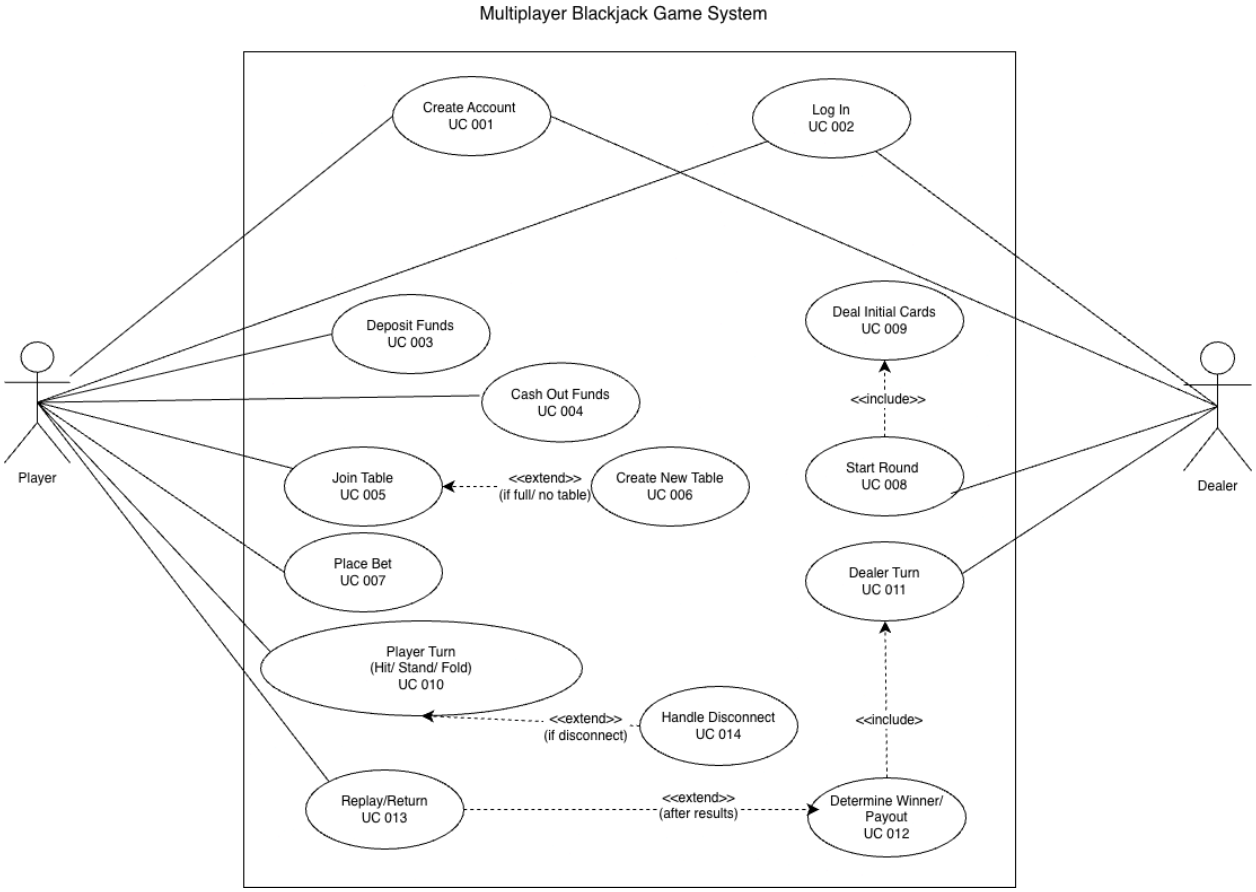
Extensions or Alternate Flows:

- Disconnect during Player's turn, server automatically disconnects and performs a fold action. Next turn moves to the next player.
- Disconnected during the round, the server removes the player from the table.
- Dealer disconnects, round is paused and waits for the next dealer, players are notified.

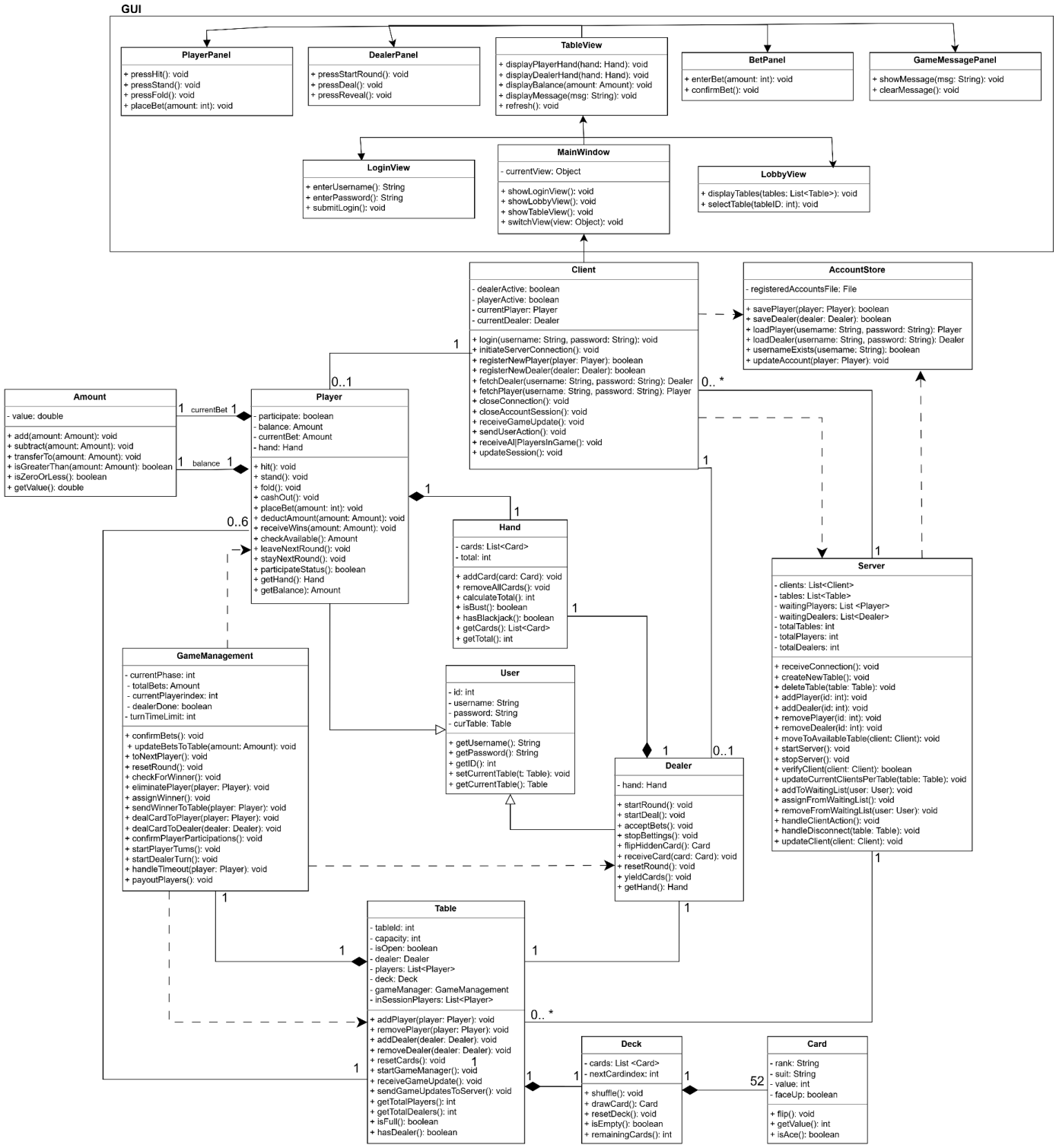
Exceptions: multiple simultaneous disconnects. The server will maintain table stability and logs events.

Related Use Cases: UC-010, UC-012, UC-013, UC-005

UML Use Case Diagram:



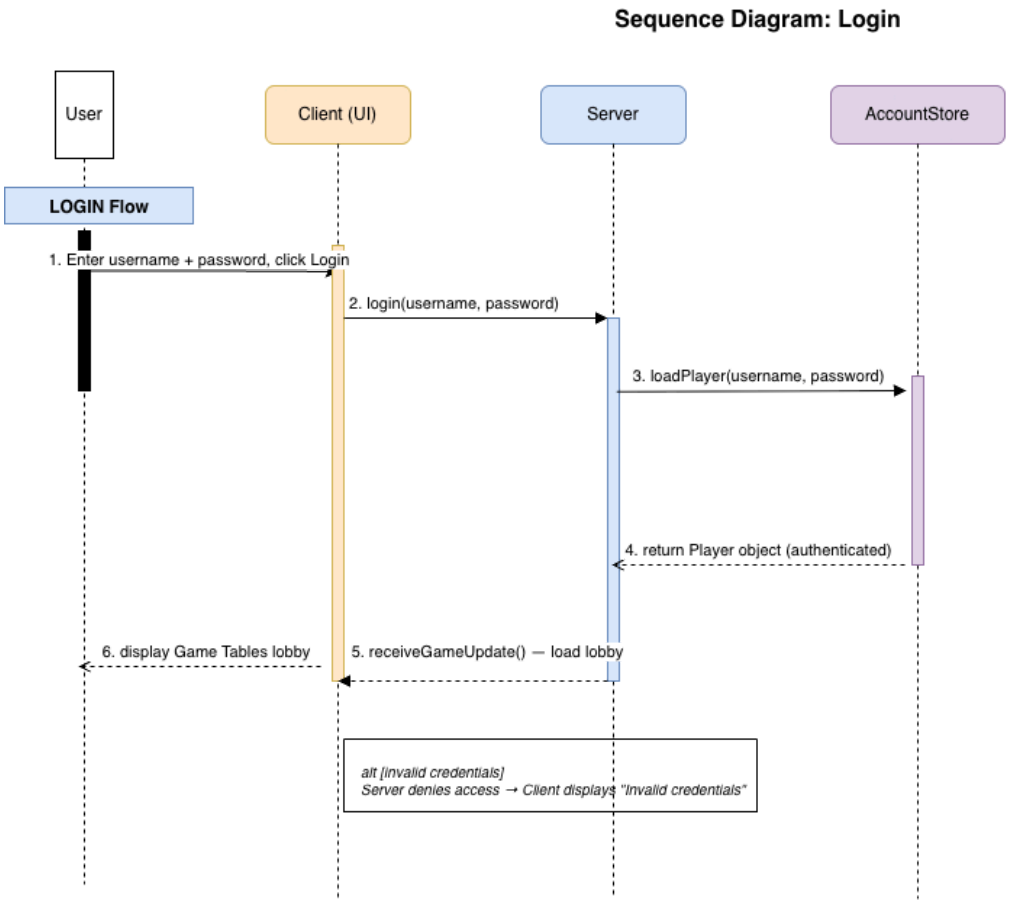
Class Diagram:



Sequence Diagram:

- **Use Case ID: 001 – Create Account**

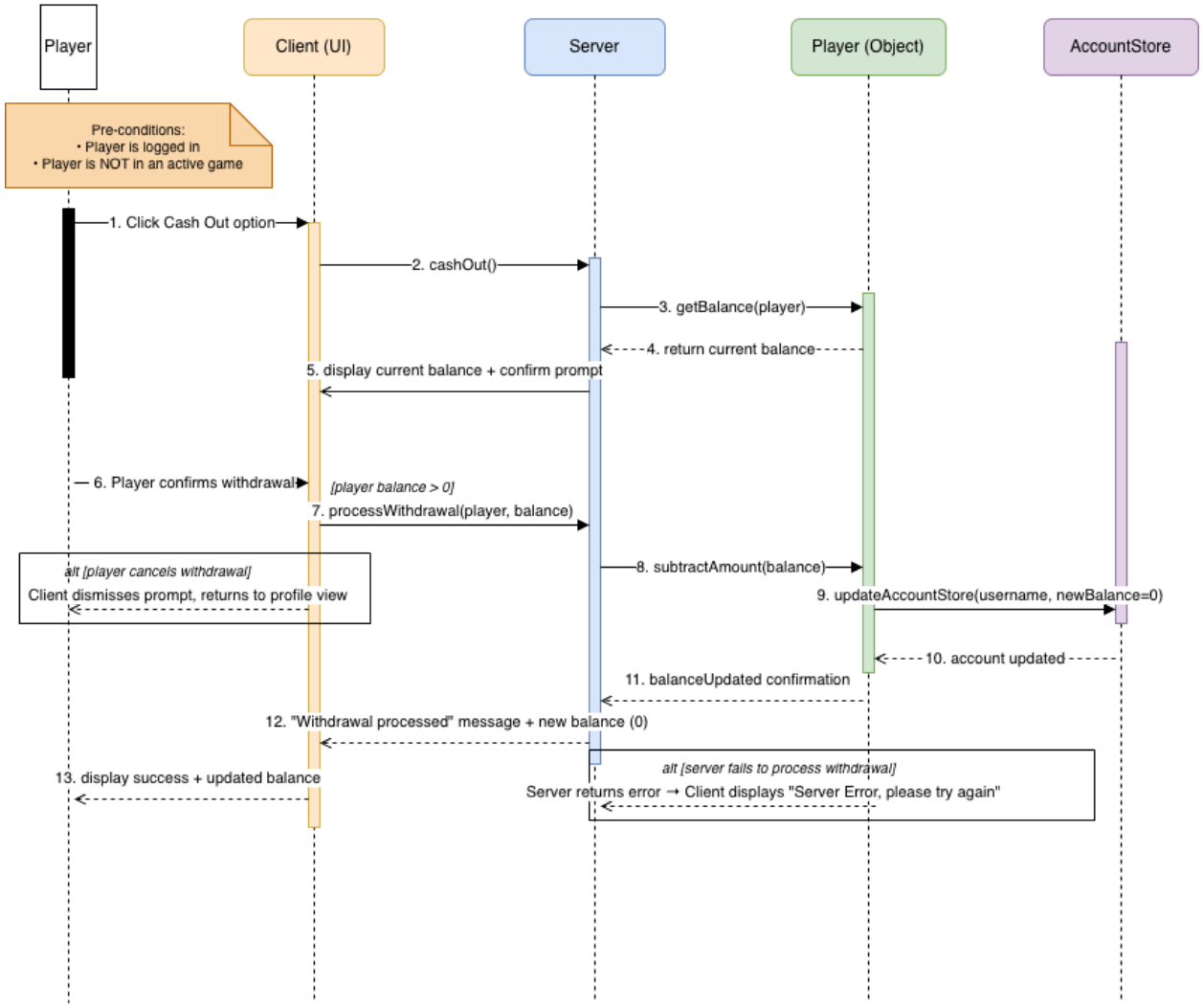
- **Use Case ID: 002 – Log In**



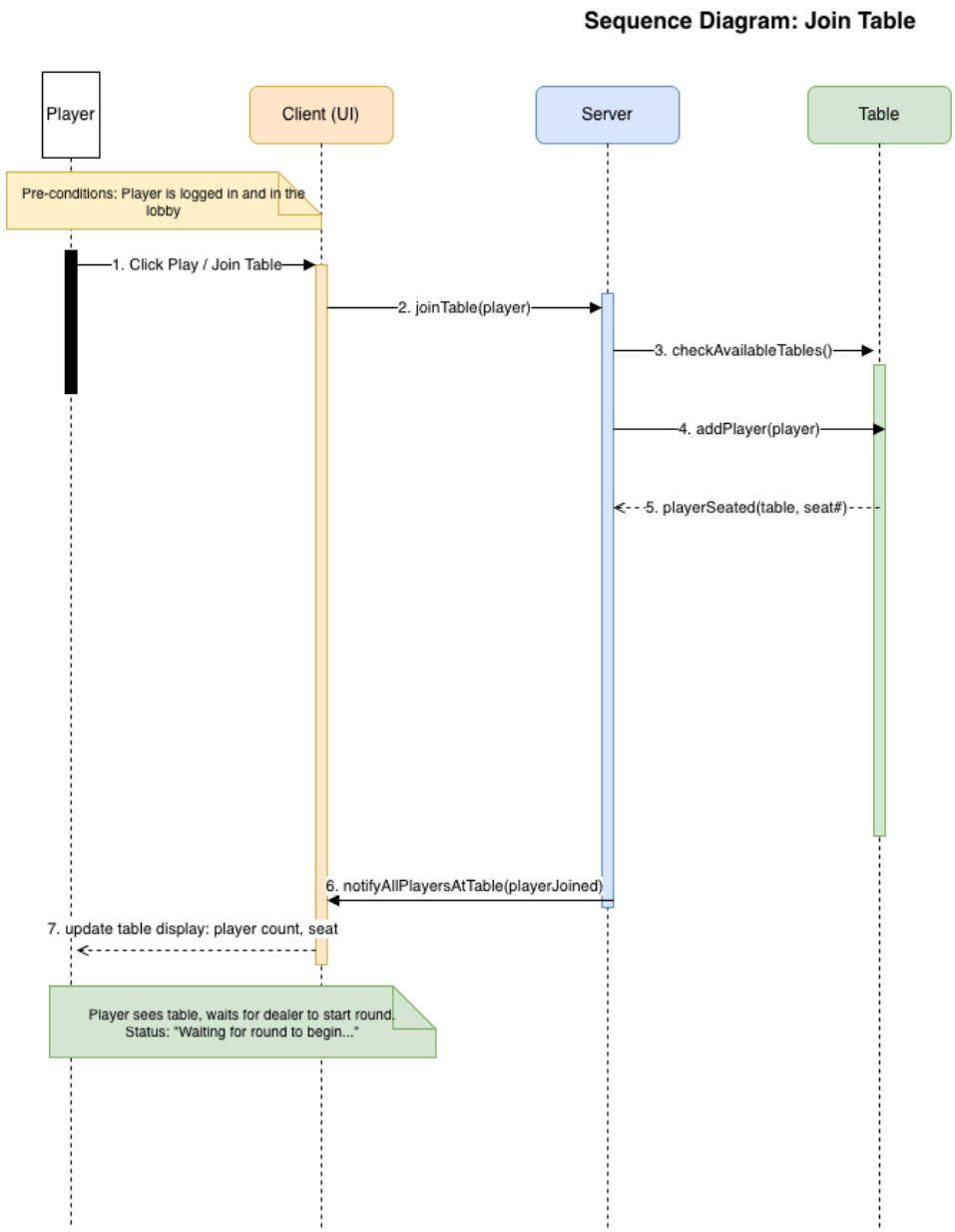
- **Use Case ID: 003 – Deposit Funds**

- **Use Case ID: 004 – Cash Out Funds**

Sequence Diagram: Cash Out Funds

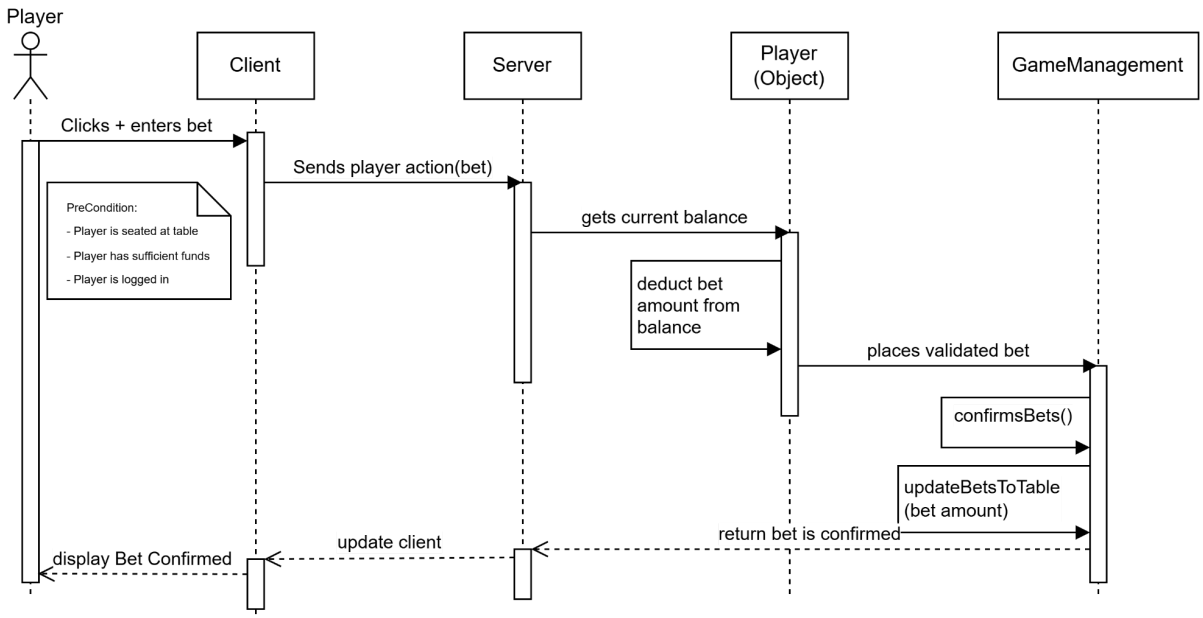


- **Use Case ID: 005 – Player Joins Table**



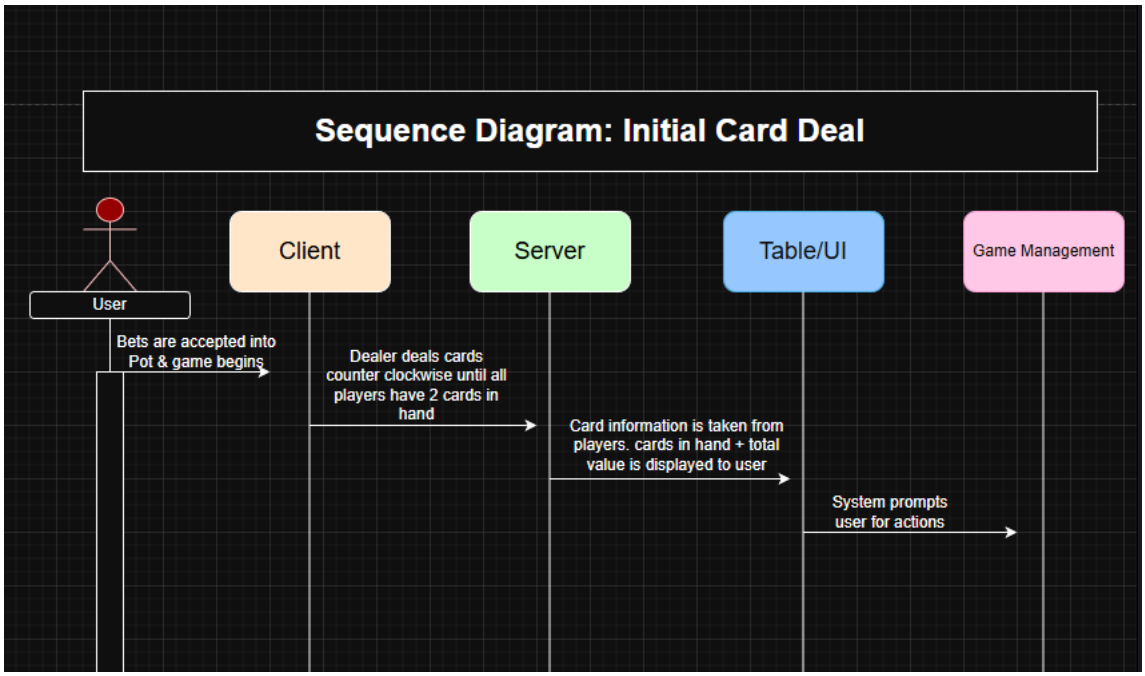
- **Use Case ID: 006 – Create New Table**

- **Use Case ID: 007 – Place Bet**

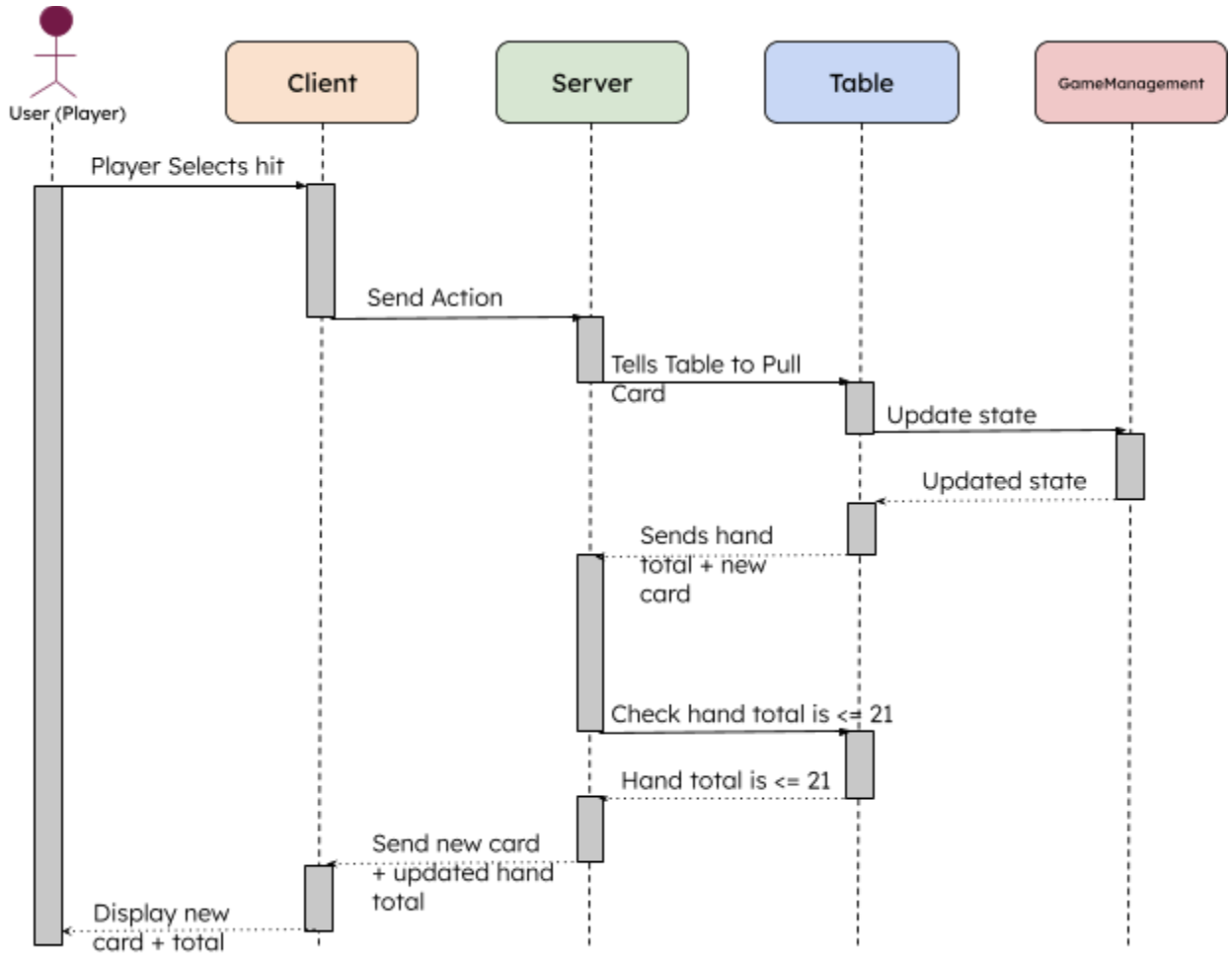


- **Use Case ID: 008 – Start Round (Dealer Begins Game)**

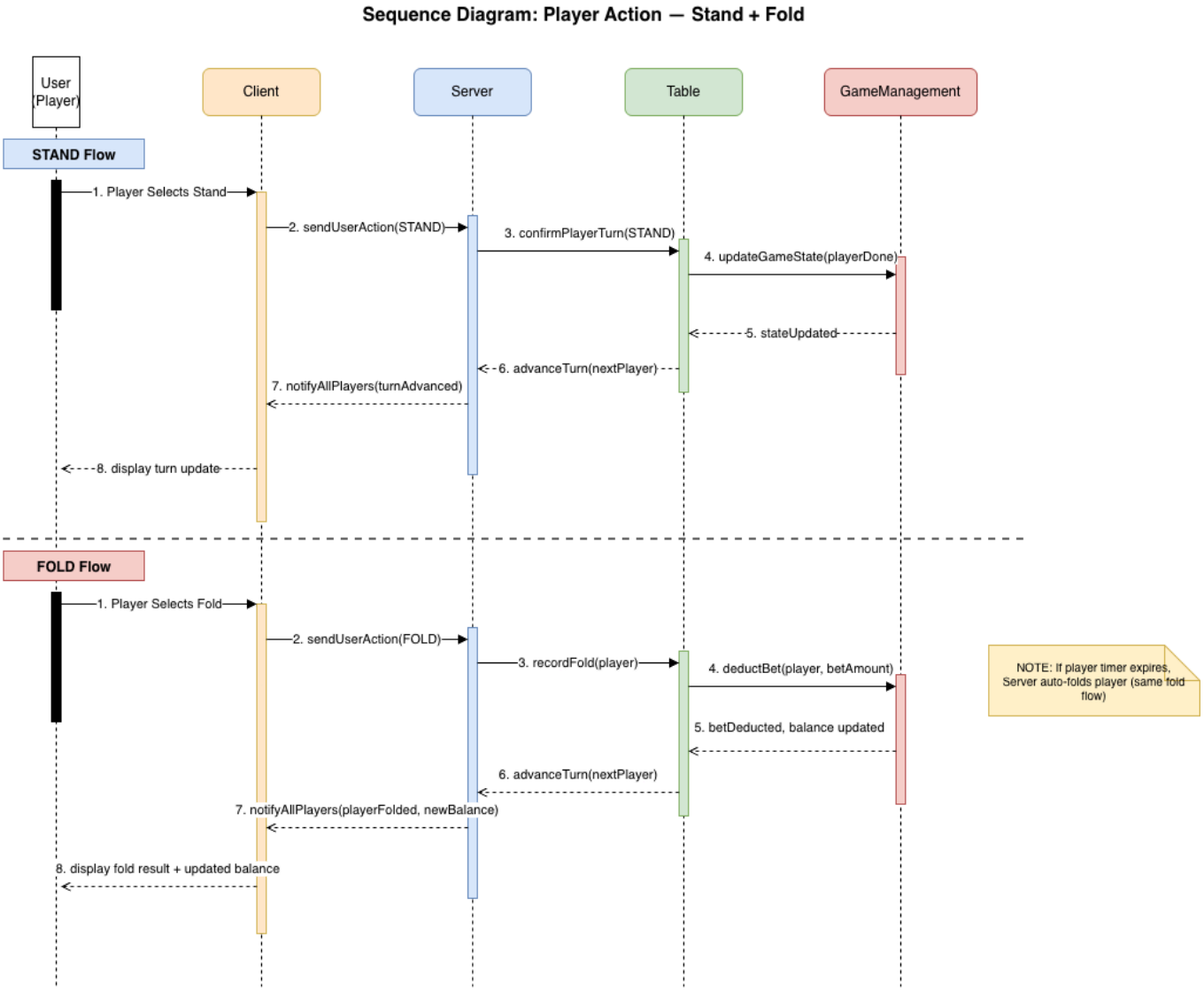
- **Use Case ID: 009 – Deal Initial Cards**



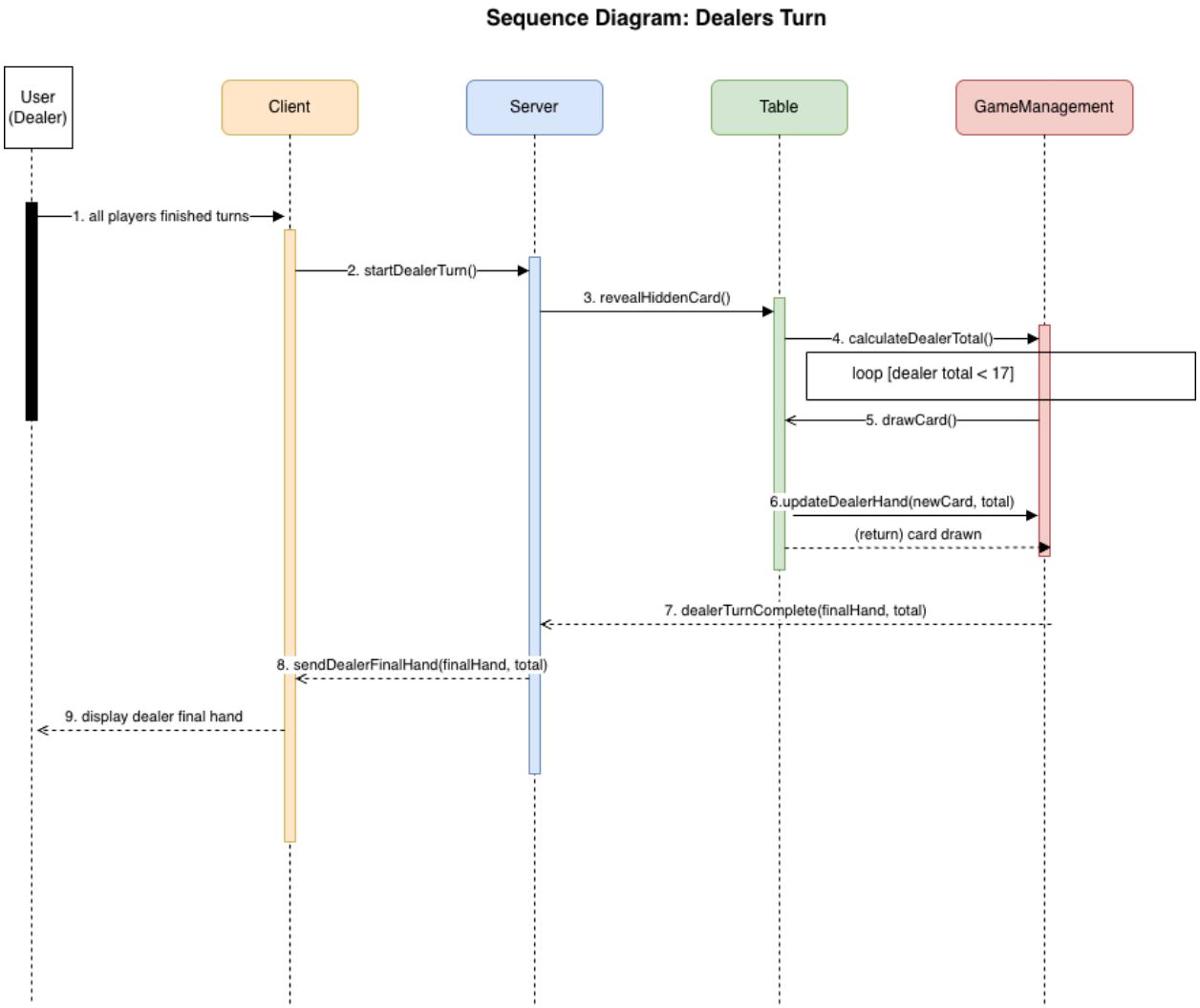
- **Use Case ID: 010 – Player Turn(HIT)**



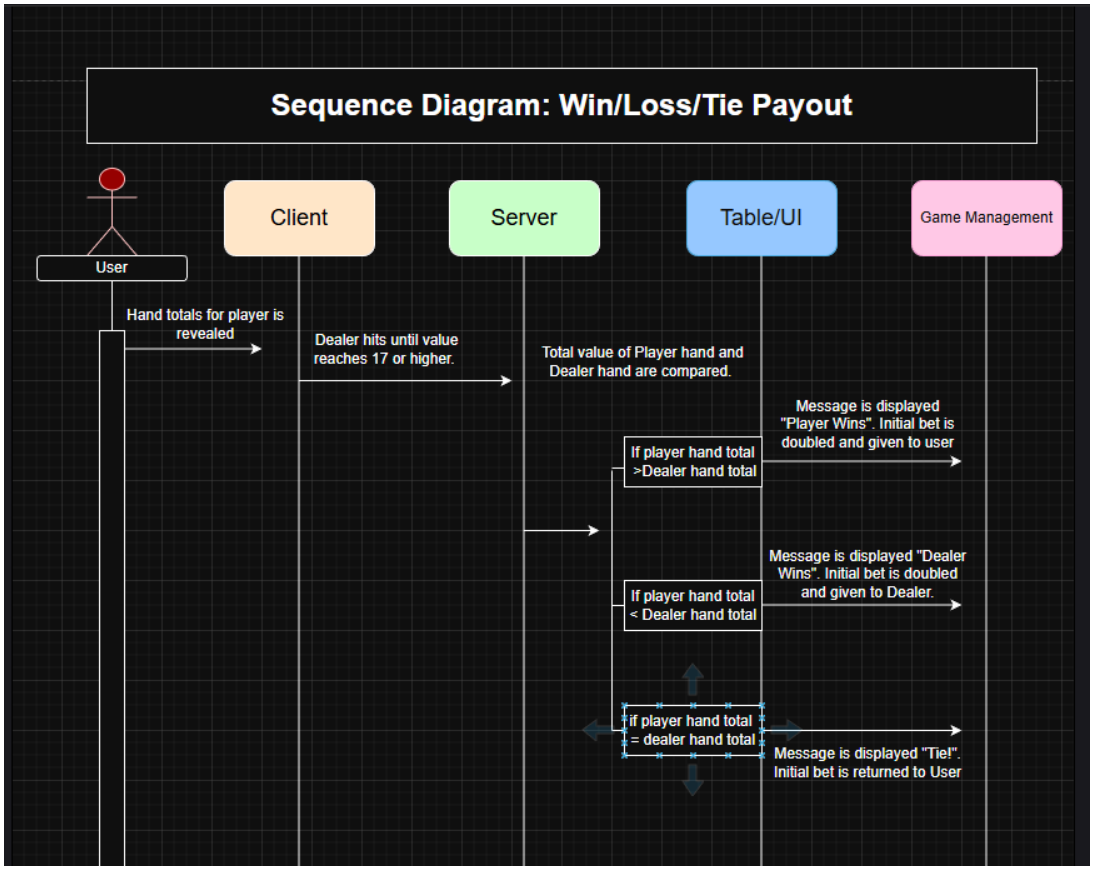
- **Use Case ID: 010 – Player Turn(STAND/FOLD)**



- **Use Case ID: 011 – Dealer Turn**



- **Use Case ID: 012 – Determine Winners / Payout**



- **Use Case ID: 013 – Replay Round / Return To Menu**

- **Use Case ID: 014 – Handle Player/Dealer Disconnect**

Github repository

https://github.com/nikkiong/Black-Jack